

An Apparatus for Verifiable AI Research

A framework that lets language models do the research — while refusing to *trust* them. Every claim is inventoried in a graph; cited claims bind to registered sources, and missing evidence receives an explicit `NO-SOURCE` verdict instead of silence. A sample of claims is checked by mechanically confined verifiers, the adversarial review is written by a model from a different vendor, and the whole process is recorded in a hash-chained, externally anchored log where any tampering with history is detectable.

3 validation gates 6 verdicts 850 audit events production-proven 08/2026

- | | |
|---|--|
| 1. Why the apparatus exists | 6. Verdicts and sampling |
| 2. System in context (C4) | 7. Academic standards |
| 3. Pipeline: preflight + 12 phases, 3 gates | 8. Blind adversarial review |
| 4. Containers and artifacts (C4) | 9. Empirical evidence: the wearables run |
| 5. The evidence chain | 10. The short profile (experimental) |
| | 11. Lessons from operation |

01 · MOTIVATION

Why the apparatus exists

A language model can assemble in hours a research report that would take a team weeks. Just as easily, it can produce a convincing-sounding claim its source does not support — or a source that does not exist. For decision-making (and for academic work) such output is worthless until you can *demonstrate what was verified, what was not, and who verified it*.

The apparatus answers with three principles:

Fail-closed. Every check fails on doubt; nothing passes silently. A verdict that violates the schema halts the run; a missing artifact blocks the gate.

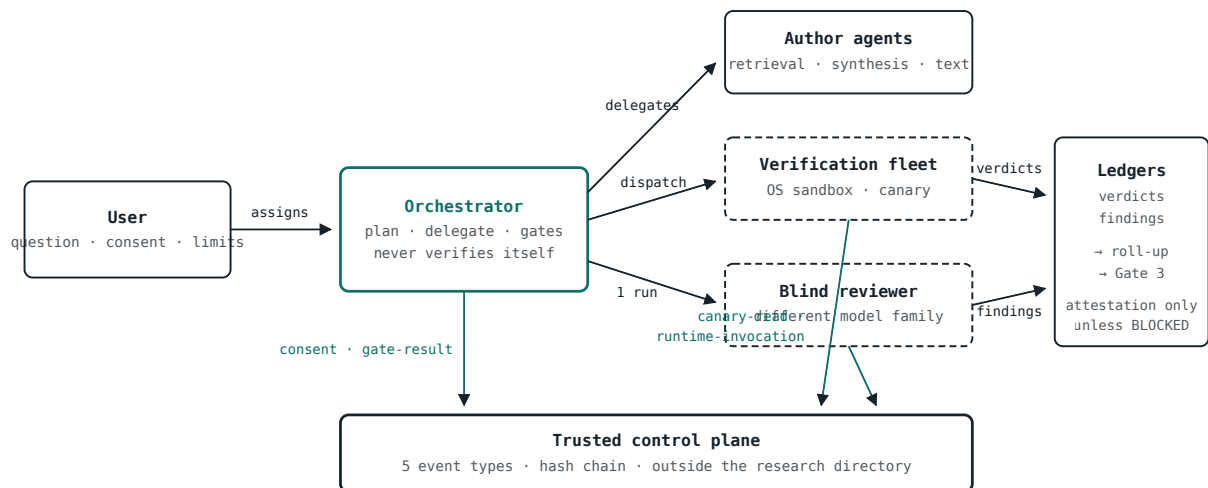
Mechanical distrust, not promises. A verifier is not "instructed not to read other files" — it runs inside a deny-by-default operating-system sandbox, and under the same

sandbox profile an active probe (canary) measures that reading a selected forbidden path actually returns `denied`. The model's promise doesn't count; the measured result does.

An audit trail instead of trust in the author. The author of the research is a model — so quality is not judged by the author but by independent instances: a verification fleet over a sample of claims, and a reviewer from a different model family, both recorded in the chained log.

02 · C4 – CONTEXT

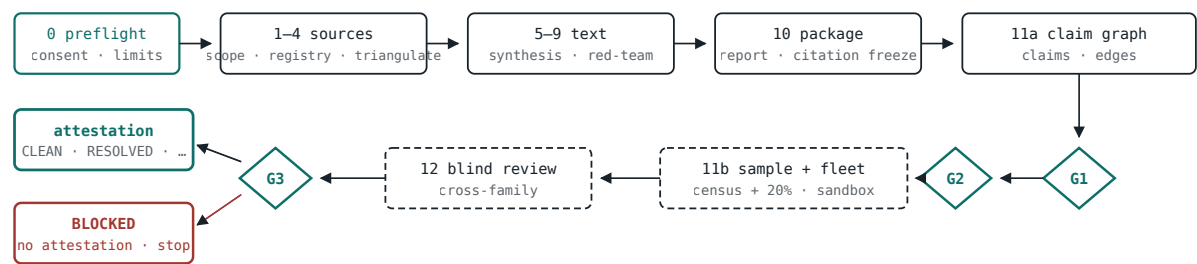
System in context



C4 Context. The orchestrator delegates but never verifies — each control role runs in a separate instance (dashed = mechanically confined). Exactly five trusted event types are written to the control plane (consent-choice · author-session · runtime-invocation · canary-read · gate-result); verdicts and findings flow into append-only ledgers from which Gate 3 derives its outcome — report attestation is conditional (BLOCKED forbids it).

03 · PROCESS

Pipeline: preflight + 12 phases, 3 gates



Phases and gates. Phase 0 (preflight with consent and cost limits) precedes the research proper in phases 1–10. Phase 11 starts by building the complete claim graph; only then do gates G1 (`sanity_pass`) and G2 (`validate_report`) run, followed by the deterministic sample and the sandboxed fleet. After the blind review (phase 12), G3 branches: the states `CLEAN` · `RESOLVED` · `CAPPED` · `NOT-RUN` lead to a written attestation; the state `BLOCKED` forbids finalization — the run ends without an attestation.

GATE	SCRIPT	WHAT IT ENFORCES
Gate 1	<code>sanity_pass.py</code>	structural integrity, out-of-scope leakage (contamination), coverage and distribution of generalizability tags
Gate 2	<code>validate_report.py</code>	tier derived from distinct cited sources vs. the declared tier, consistency of inline citations with the registry and the claim graph
Gate 3	<code>review_gate.py</code>	completeness of verification and review, adjudication of findings, hash consistency — writes the attestation only outside the <code>BLOCKED</code> state

Containers and artifacts

Authoring plane

- `claim_sampler.py` — claim graph and deterministic sample
- `source_registry.py` — source registry (schema r4: appraisal, COI, retractions)
- `verify_citations.py` — DOI/URL checks before the freeze
- artifacts: `report.md`, `sources.jsonl`, `claims.jsonl`, `claim_evidence_edges.jsonl`

Verification plane

- `verifier_dispatch.py` — sandbox profile + canary + dispatch
- `fleet_runner.py` / `fleet_ledger.py` — sequential fleet, crash-resume, append-only state
- `verify_ledger_builder.py` — attempt ledger → roll-up → evidence packet
- artifacts: `qa_raw/*` (content-addressed), `claim_verify.jsonl`

Review plane

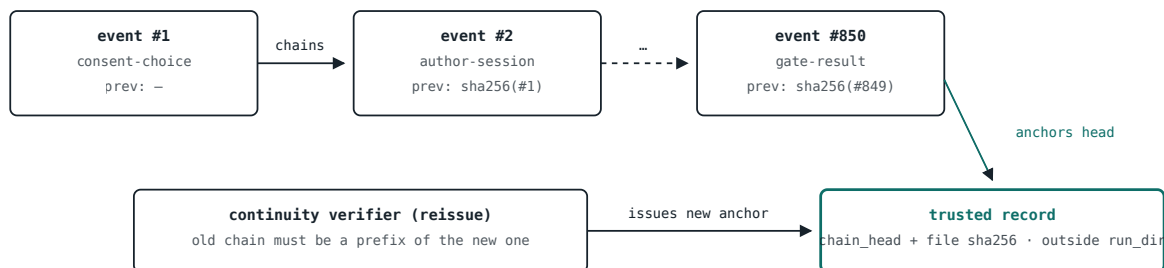
- `review_bundle.py` — six-member bundle in pinned order
- `review_dispatch.py` — confined reviewer run (Codex)
- `blind_ledger_builder.py` — findings → adjudicable ledger
- `qa_manifest_builder.py` — hash manifest for G3

Trust plane

- `trusted_control_plane.py` — hash-chained event log
- `record_issuer.py` — continuity verifier (prefix anchoring)
- `qa_schemas.py` — fail-closed validation of all schemas
- artifact: `trusted-control-record.json` **outside the run directory**

The protected core consists of **eight globally frozen scripts** (`claim_sampler`, `validate_report`, `sanity_pass`, `review_gate`, `qa_schemas`, `source_registry`, `verifier_dispatch`, `trusted_control_plane`); the v2.2 design documents add three further protected dependencies (`source_evaluator`, `verifier_envelope`, `review_bundle`). Changing any of their contracts would invalidate in-flight research, so new capabilities are added exclusively as new scripts alongside them.

The evidence chain



Tamper-evident, externally anchored evidence. Changing any earlier event breaks the digests of everything after it. The anchor lives outside the research directory; routine reissue is done by `record_issuer.py`, which first checks that the old chain is a literal prefix of the new one. Other trusted orchestrator tools also write the anchor (`gate_result_recorder.py` after gate writes, `control_plane_reset.py` during a deliberate, backed-up reset) — the model therefore rests not on physical immutability but on every intervention being either a prefix extension or an auditable operation by a trusted writer outside the run's reach; anything else the anchor exposes.

06 · MEASUREMENT

Verdicts and sampling

A verifier receives the claim, its binding to a source, and the full source text — and returns exactly one of six verdicts. For `SUPPORTED`, `verify_shim.py` checks that the quoted excerpt is literally contained in the source text; `qa_schemas.py` then enforces, fail-closed, a non-empty excerpt, locator, support type, and the source and content hashes.

■ SUPPORTED ■ PARTIAL ■ UNSUPPORTED ■ CONTRADICTED ■ SOURCE-UNAVAILABLE
■ NO-SOURCE

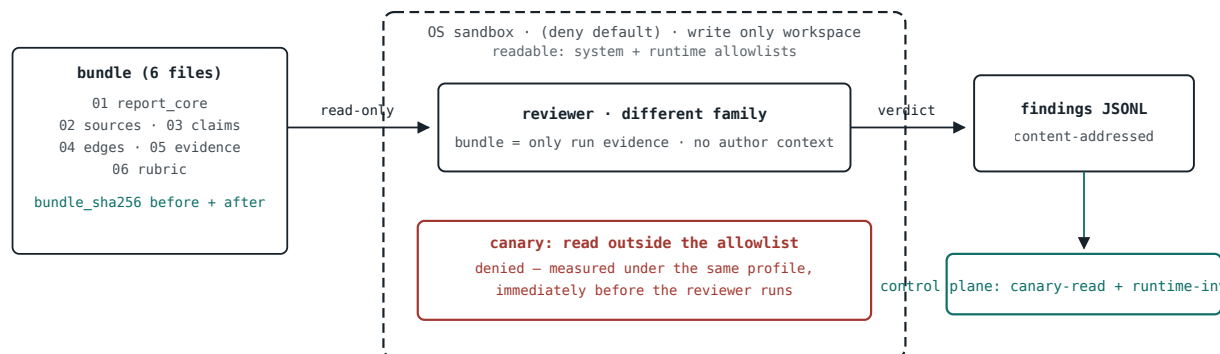
The sample is not random. **A full census** (no sampling) covers every formal `finding` and every unit flagged `executive_summary`, `high_confidence`, `numeric`, `safety`, `legal`, `decision_critical` or `recommendation_support`; the high-stakes profile additionally saturates `high_stakes_assertion` units. Only the *remaining* ordinary assertions are sampled at a deterministic 20% with per-section quotas. The "randomness" of selection is a sort key, `sha256(report_core_sha256 + ":" + claim_hash)` — selection is reproducible and bound to the exact report version, with no RNG. A pair's summary composes conservatively: the worst verdict wins (`CONTRADICTED` → `UNSUPPORTED` → `NO-SOURCE` → `PARTIAL` → `SOURCE-UNAVAILABLE` → `SUPPORTED`).

Academic standards inside the apparatus

The apparatus does not claim to be a formal systematic review — it adopts the accounting and appraisal discipline of these standards and applies them as binding lenses, not as ticked-off checklists.

STANDARD	WHERE IT LIVES IN THE APPARATUS
PRISMA 2020 / PRISMA-S (in spirit)	logged queries for every tool, a recorded reason for every excluded source, a flow table generated by <code>source_registry.py prisma</code> ; the output is not presented as a systematic review
GRADE (simplified)	High/Medium/Low with downgrade domains (risk of bias, inconsistency, indirectness, imprecision, publication bias) at claim level
Triangulation	a tier floor on the number of <i>independent</i> sources per claim; independence by publisher, author and funding lineage (<code>independence_group</code> in the edge graph) — the blind review probes it explicitly
COI / funding	a <code>funding_coi</code> field in the registry + a vendor-funded dependency check in the critique phase; AACODS for grey literature
Generalizability	mandatory     tags on every finding; a gate checks their distribution, the reviewer checks tag width against the strength of evidence
Domain lenses	Kitchenham SLR (IT), Tranfield (management), CONSORT-AI / TRIPOD+AI / STARD-AI and others for AI-in-health — as appraisal lenses, not certifications

Blind adversarial review



Blindness as mechanics, with declared boundaries. The reviewer never receives the authoring conversation, and the bundle is the only research-run evidence exposed; alongside it the sandbox leaves only system and runtime allowlists readable, with writes confined to the working directory. The canary is an active probe of a selected forbidden path, launched under the same sandbox profile in the immediately preceding invocation — it proves the profile is enforced, not universal blindness. The ban on web access and fact-checking is imposed by the rubric (a role rule, not mechanics). Adjudication of findings has a budget (1 full round + 2 confirmation rounds), and every acceptance forces a fix, re-verification, and a fresh bundle against the new hash.

09 · EVIDENCE

Empirical evidence: the wearables run

The apparatus is not a proposal — in August 2026 it completed a **full-scale production dataset and control exercise** over a research report on the clinical validity of wearables. The resulting manifest honestly remained in the **BLOCKED** state (see below); all numbers come from the run's own audit artifacts.

316

verified claim ↔ source
pairs (census classes
+ 20% sample)

421

sandboxed verifier
invocations in the
control plane

43 %

verification coverage
(floor of 138/316;
rounded down)

14

findings by the cross-
family reviewer (6
Critical, 8 Major)

850

hash-chained events
in the trusted control
plane

■ 138 ■ 48 ■ 27 ■ 1 ■ 23 ■ 79

The single CONTRADICTED result (0.32 %) is drawn at a minimum visible width so it does not vanish.

Gates: Gate 1 PASS Gate 2 PASS Gate 3 BLOCKED — and the last one is the most valuable evidence of all. Finalization refused to issue an attestation and printed **1,085 validation errors across documented classes**: 913 non-canonical JSON records in input artifacts, 156 unresolved pairs, an exhausted review budget (round one was killed by a Gatekeeper quarantine on the host), an incomplete review ledger for the blocked round, missing flag-audit tooling, three claims below the triangulation floor, and two projection/re-derivation mismatches. The apparatus thus *refused to rubber-stamp its own run* rather than conceal the defects — for a framework whose whole point is distrust, that is a feature, not a failure.

Cross-family independence, proven in data: author `claude-sonnet-5` (family `claude`), reviewer `gpt-5.6-sol` (family `openai`), `read_confinement: os-sandbox-allowlist` — all recorded in the manifest and the control plane, not in the author's own statement.

10 · LIGHTWEIGHT MODE

The short profile (experimental)

The short profile is an **experimental extension outside the active SKILL.md** (which knows a single mode, MAXIMUM): its v2 specification was produced on 2026-08-27 after a blind review of its own (version 1 received 4 Critical and 4 Major findings) and it has been deployed for supplementary runs supporting this project.

Keeps

- the protocol up front (scope, criteria, queries)
- the full source registry incl. appraisal and COI
- the claim graph + deterministic sample (unchanged)
- gates G1 and G2, byte-exact
- an append-only spot-check ledger bound to the report hash

Drops — and declares

- the sandboxed verification fleet — **the spot-check verifier runs without an OS sandbox**
- the blind adversarial review
- the hash-chained control plane and the G3 attestation
- instead: a NOT-RUN – short profile declaration written into the report by a fail-closed finalizer

Decision-critical claims (those whose refutation would change a go/no-go call, a financial threshold, or legal permissibility) receive **full verification of all their pairs** even in the short profile — never a sample. An adverse verdict is never resolved by prose — only by narrowing or removing the claim, replacing or removing the evidence, or downgrading confidence; every such edit forces a fresh re-check against the new report hash, and removals are reported separately by the finalizer (resolved-by-removal).

11 · OPERATIONS

Lessons from operation

A false success is worse than a crash. A smoke probe standing in for a verifier returned a hardcoded verdict — it was caught only by the schema, which rejected "no source" for a pair that had one. Since then the rule stands: validate every intermediate result against a schema, and never trust the exit code (a headless run can exit zero with the work undone).

The environment is part of the proof. A Gatekeeper binary quarantine managed to block the review in a way that looked like a runtime timeout; confinement is therefore proven by a probe on the same machine and build where the production run executes.

Verifiers oscillate. Borderline claims receive different verdicts from independent runs; convergence is guaranteed by the append-only ledger (history is never rewritten) and by the rule that the final round before finalization makes no edits.

The safeguards bind the apparatus's own author too. The phase-0 cost gate blocked the run's own finalization until the re-estimate was computed methodically; the undefined estimation unit is a documented defect for the next version.

English edition of the Czech original (v3), translated by the same orchestrator session; content identical, no audience-specific additions

the Czech original passed a blind Codex review of v1 (4 Critical, 7 Major, 3 Minor; protocol: docs/reviews/2026-08-28-framework-presentace-oponentura-codex.md) and a confirmation round of v2 (9 resolved, 5 partially → resolved in v3); v1 is preserved in the artifact's version history, v2 = git e6bf8a9